



**DEPARTMENT OF COMPUTER &
SOFTWARE ENGINEERING
COLLEGE OF E&ME, NUST,
RAWALPINDI**



SUBMITTED TO:

Snr Lec. Anum Abdul Salam

LE Hira Irshad

SUBMITTED BY:

Student Names: Muhammad Husnain (507214)

Abdullah Aqeel Khan (501845)

Muhammad Usman Irshad (533076)

DE- 46 Dept C&SE Syn B

Contents

Problem Statement	4
Introduction.....	5
Technical Implementation Details	6
List of Functionalities:	6
Functionalities in Depth:.....	7
1. Vector3D.java (Math Utility)	7
2. DroneState.java (Data Container).....	8
3. Controller.java (Movement Control).....	8
4. Drone.java (Physics Integration)	8
5. FormationManager.java (Swarm Cohesion)	9
6. CollisionAvoidance.java (Safety System)	9
7. CommunicationModule.java (Networking).....	9
8. Environment.java (World Boundaries)	10
9. Simulator.java (The Engine).....	10
10. Main.java (Mission Manager)	11
11. DroneGUI.java (3D Visualization).....	11
12. Logger.java (Data Recorder).....	11
Application of OOP Pillars.....	12
Project Execution Flow:	12
1. System Initialization	12
2. The Real-Time Simulation Loop	13
1. Sense (Communication):.....	13
2. Think (Swarm Logic):	13
3. Act (Control):	13
4. Integrate (Physics):	13
3. Mission Phase Progression.....	13
• Phase 1: GO_TO_TARGET	13
• Phase 2: SPRAYING:.....	13
• Phase 3: RETURNING:.....	13

• Phase 4: DONE:	13
4. Data Logging and Termination	13
• Real-time Recording:	13
• Finalization:.....	13
High Level Control Flow:	14
Summary of Mathematical Formulas	14
OUTPUT OF PROGRAM: Spraying:.....	16
UML Class Diagram:	17
Limitations:	17
1. Communication Realism.....	17
2. Physical and Environmental Constraints	17
3. Flight Dynamics & Hardware	18
4. Swarm and Mission Logic	18
5. FILE I/O	18
Conclusions:	18
References:	19

Problem Statement

Modern agriculture increasingly relies on fleets of autonomous drones for tasks such as crop spraying, monitoring, and mapping. However, coordinating multiple drones in real-world environments presents significant challenges due to communication delays, unpredictable weather, and collision risks. Manual approaches to agricultural spraying are labor-intensive, time-consuming, and prone to inefficiencies, while hardware-based drone testing is costly and unsafe without prior validation. Existing systems often lack realistic modeling of swarm behavior, communication loss, and environmental constraints, limiting their reliability. Real-world implementation further faces challenges such as sensor inaccuracies, battery limitations, and dynamic obstacles like trees or power lines. Designing and testing swarm intelligence algorithms directly on hardware can lead to expensive failures and safety hazards. Therefore, a simulation-based solution provides a controlled, cost-effective, and safe environment to evaluate drone coordination strategies. By integrating physics-based models, communication uncertainty, and swarm control logic, the simulator allows researchers to analyze performance without risking equipment or human safety. This approach ensures reproducibility, scalability, and flexibility, making it an essential step toward deploying autonomous drone fleets in precision agriculture.

Introduction

This project presents a physics-based **Autonomous Drone Fleet Simulator** specifically engineered for precision agricultural spraying missions. The system utilizes a modular Java architecture where the **Simulator** engine manages realistic physics integration and environmental constraints based on parameters loaded from external configuration files, ensuring the simulation is flexible and not limited by hard-coded values. Each drone operates under a sophisticated state-machine logic that orchestrates transitions between target navigation, a precise ten-second spraying phase, and an autonomous return to the initial base. To ensure collective safety and efficiency, the fleet integrates **swarm intelligence** algorithms, including consensus-based formation control to keep the group together and repulsive-force collision avoidance to prevent mid-air impacts. The entire mission is visualized through a comprehensive **3D JavaFX interface**, which maps mathematical physics to a visual space and utilizes a dynamic particle system to represent pesticide application in real-time.

Technical Implementation Details

- **Programming Language:** The project is developed in **Java**, utilizing its strong object-oriented features to handle complex concurrent simulations and mathematical modeling.
- **Development Tools:**
 - **Eclipse 2025:** Used as the primary Integrated Development Environment (IDE) for code management and debugging.
 - **JavaFX SDK:** Employed to render the real-time **3D Graphical User Interface**, providing hardware-accelerated visualization of the drone swarm.
- **Project File Structure:** A highly modular, **object-oriented architecture** where each class is mapped to a distinct physical or computational process. This ensures that physics logic (Drone/State) is decoupled from mission logic (Main/Controller) and visualization (GUI).

List of Functionalities:

- ☐ Autonomous Mission Execution
- ☐ Waypoint Navigation
- ☐ Timed Agricultural Spraying
- ☐ Automated Return-to-Base
- ☐ Newton-Euler Physics Integration
- ☐ 3D Rotational Dynamics
- ☐ Swarm Cohesion and Formation Control
- ☐ Repulsive Collision Avoidance
- ☐ PD (Proportional-Derivative) Flight Control
- ☐ Proximity-based Communication Filtering
- ☐ Stochastic Message Loss Simulation
- ☐ Environmental Boundary Clamping
- ☐ Soft-wall Repulsion Logic
- ☐ External Configuration Loading

- ☐ Real-time 3D JavaFX Visualization
- ☐ Visual Particle Spray System
- ☐ Historical Flight Trajectory Tracking
- ☐ High-resolution Telemetry Logging (CSV)
- ☐ Mission Success Metrics Reporting

Category	Key Classes	Responsibility
Orchestration	Main.java, Simulator.java	Mission state control and simulation stepping.
Agent Logic	Drone.java, Controller.java	Individual flight physics and PID control.
Social Behavior	FormationManager.java, CollisionAvoidance.java	Swarm cohesion and safety protocols.
Data & Visuals	DroneGUI.java, Logger.java	3D rendering and telemetry export.

Functionalities in Depth:

1. Vector3D.java (Math Utility)

- a) This class serves as the mathematical foundation for all 3D physics calculations.
- b) It stores the primary spatial components: the x, y, and z coordinates.
- c) It provides standard vector operations such as addition and subtraction.
- d) Includes scalar multiplication and division methods for scaling physical forces.
- e) Implements the dot product, which is used for projection and energy math.
- f) Features a cross product method, essential for calculating rotational torque.
- g) Contains a magnitude function to determine the drone's exact speed or distance.
- h) Offers a normalization tool to create unit vectors for direction handling.
- i) Utilizes "in-place" operations to modify existing vectors and save memory.
- j) Every physical movement in the simulation relies on this core utility class.

2. DroneState.java (Data Container)

- a) This class acts as a central hub for storing a single drone's current status.
- b) It tracks translational motion through position, velocity, and acceleration.
- c) It manages a 3x3 rotation matrix to keep track of the drone's orientation.
- d) Stores rotational dynamics, including angular velocity and angular acceleration.
- e) Holds physical constants like mass, inertia, and the drag coefficient.
- f) Records real-time control inputs such as vertical thrust and motor torque.
- g) Specifically labels external forces like formation, repulsion, and environment.
- h) Serves as the primary data source for both the Logger and the 3D GUI.
- i) Allows the physics engine to access all drone properties in one object.
- j) It represents the "physical identity" of every agent flying in the swarm.

3. Controller.java (Movement Control)

- a) This class acts as the "pilot" by deciding how each drone should move.
- b) It implements a Proportional-Derivative (PD) control loop architecture.
- c) It calculates the "error" distance between the current position and the target.
- d) Derives a "desired acceleration" to close the gap to the next waypoint.
- e) Automatically compensates for gravity so the drone can hover steadily.
- f) Determines the exact vertical thrust required to maintain or change altitude.
- g) Applies angular damping to prevent the drone from spinning out of control.
- h) Adjusts the torque based on the drone's current tilt and rotation matrix.
- i) Converts high-level navigation goals into low-level physical force values.
- j) It is essential for translating mission logic into smooth, stable flight.

4. Drone.java (Physics Integration)

- a) This class represents an individual drone unit within the simulation world.
- b) It links the data in DroneState with the logic found in the Controller.
- c) The integrate() method is the heart of its physical movement engine.
- d) It applies Newton's Second Law ($F=ma$) to calculate new motion steps.
- e) It calculates the sum of gravity, thrust, drag, and all swarm-based forces.
- f) Updates the drone's position and velocity vectors for every time-step.

- g) Includes the math required to update the 3D rotation matrix over time.
- h) Manages a list of neighbors that the drone can "see" and interact with.
- i) Assigns a unique ID to each drone for organized tracking and logging.
- j) It bridges the gap between abstract physical laws and the simulation loop.

5. FormationManager.java (Swarm Cohesion)

- a) This class implements the "social" behavior of the drone fleet.
- b) It uses a consensus algorithm to ensure the drones fly as a group.
- c) Calculates an attraction force based on where neighbors are positioned.
- d) Analyzes neighbor velocities to ensure the whole fleet moves at once.
- e) Prevents individual drones from drifting away during long-distance flight.
- f) Uses position and velocity coupling gains for tuning.
- g) Only considers drones that are within a specific communication range.
- h) The resulting force pulls the drone toward the average center of the group.
- i) Ensures the fleet maintains a cohesive, organized, and professional structure.
- j) Without this class, drones would act as isolated and uncoordinated units.

6. CollisionAvoidance.java (Safety System)

- a) This class acts as a vital safety layer to prevent mid-air drone crashes.
- b) It uses a repulsive force model based on a mathematical Inverse-Square Law.
- c) It generates a force that pushes drones away if they get too close.
- d) The logic only activates when a neighbor enters a defined "danger zone."
- e) The force magnitude increases exponentially as the distance between drones shrinks.
- f) It iterates through all nearby drones to check for any immediate impact risks.
- g) Provides a safety buffer that can override the formation's cohesion force.
- h) Ensures that drones maintain a minimum safe distance during all phases.
- i) Includes safety checks to prevent errors when drones are extremely close.
- j) This logic is critical for maintaining fleet integrity in tight mission areas.

7. CommunicationModule.java (Networking)

- a) This class simulates the wireless networking environment for the swarm.
- b) It determines which drones are close enough to exchange their sensor data.

- c) Uses a "Communication Range" parameter to filter out distant drones.
- d) Introduces "Stochastic Loss" to mimic real-world signal interference.
- e) A random probability check determines if a message is lost or received.
- f) This adds a layer of realistic uncertainty to the swarm's behavior.
- g) Provides the getNeighbors() method used by all other swarm modules.
- h) Simulates how environmental noise can disrupt collective fleet coordination.
- i) Helps test how robust the swarm is against intermittent network failures.
- j) It bridges the gap between perfect math and imperfect real-world hardware.

8. Environment.java (World Boundaries)

- a) This class defines the 3D "bounding box" where the simulation occurs.
- b) It sets the minimum and maximum boundaries for the x, y, and z axes.
- c) Implements "Soft Walls" that push drones back if they stray too far.
- d) Applies a spring-like repulsive force near the edges of the world.
- e) Features "Hard Clamping" to physically block drones from exiting the map.
- f) Specifically prevents drones from flying below the ground level .
- g) Resets the drone's velocity to zero if it makes contact with a wall.
- h) Ensures the mission stays within the designated agricultural field area.
- i) Provides a controlled environment for safe and stable physics integration.
- j) Serves as the ultimate physical limit for the entire simulation world.

9. Simulator.java (The Engine)

- a) This is the central engine that manages the simulation's execution timing.
- b) It loads system parameters from a "config.txt" file during the startup.
- c) Initializes all drones, the communication module, and the environment.
- d) The step() method runs in a continuous loop to advance the time.
- e) It coordinates the exact sequence of communication, logic, and physics.
- f) First, it updates the neighbor lists for every drone in the fleet.
- g) Next, it calculates the formation, collision, and boundary forces.
- h) Then, it triggers the controllers to decide on the required engine thrust.
- i) Finally, it commands each drone to update its position and velocity.

- j) It connects all the independent modules into one working, rhythmic system.

10. Main.java (Mission Manager)

- a) This is the entry point of the app and the high-level "Mission Commander."
- b) It defines the mission phases: GO_TO_TARGET, SPRAYING, and RETURNING.
- c) Sets the global coordinates for the base station and the target field.
- d) Uses a state machine to track the progress of each individual drone.
- e) Implements "Velocity Steering" to allow for smooth and natural turns.
- f) Controls the spraying timer, which lasts for exactly 10 seconds per target.
- g) Applies a speed governor (clamp) to keep drones within safe flight limits.
- h) Launches the Simulator and initializes the 3D JavaFX GUI window.
- i) Determines when a drone has "arrived" based on a specific distance radius.
- j) It orchestrates the overall story and goals of the simulation from start to finish.

11. DroneGUI.java (3D Visualization)

- a) This class provides the 3D visual interface using the JavaFX framework.
- b) It maps simulation math (Z-up) to the screen's visual space (-Y up).
- c) Renders each drone as a 3D box equipped with rotating propeller spheres.
- d) Includes a particle system to visualize the pesticide spray in real-time.
- e) Features a movable 3D camera so the user can explore the flight area.
- f) Draws the X, Y, and Z axes along with a grid for better orientation.
- g) Uses an AnimationTimer to refresh the visuals 60 times every second.
- h) Visualizes "Trails" so users can see the path the drones have traveled.
- i) Displays on-screen labels showing the current mission phase for each drone.
- j) It makes complex mathematical physics easy for a human to observe.

12. Logger.java (Data Recorder)

- a) This is the data recording tool used for post-flight mission analysis.
- b) It creates a "positions.csv" file to store all raw trajectory data.
- c) Records drone IDs, 3D positions, velocities, and thrust power levels.
- d) Writes data at every simulation step for high-resolution path tracking.
- e) Helps developers debug flight paths and the performance of controllers.

- f) Generates a "metrics.txt" file once the simulation is finally closed.
- g) Summarizes the total drones used and whether the mission was successful.
- h) Uses efficient file-writing techniques to avoid slowing down the simulation.
- i) Ensures that flight data can be imported into tools like Excel or Python.
- j) It is vital for proving that the swarm behaved according to the math.

Application of OOP Pillars

- **Encapsulation:** Each class manages its own data and hides internal complexity. For example, the Drone class protects its physical properties (mass, inertia), while the DroneState class encapsulates the raw telemetry, ensuring that physical variables can only be modified through controlled simulation steps.
- **Inheritance:** This project uses a hierarchical structure to define relationships. While currently implemented through specialized classes, it allows for the potential extension of the system—such as creating a SprayingDrone that inherits base flight capabilities from a generic Drone class but adds unique mission-specific behaviors.
- **Composition:** The system follows a "has-a" design pattern to build complex agents from modular components. A Drone is not a single block of code; it **is composed of** a Controller for logic and a DroneState for physics, while the Simulator is composed of a fleet of drones and specialized manager classes.
- **Abstraction:** High-level logic is separated from low-level mathematics. The Main mission loop focuses on "what" the drone should do (e.g., PHASE = SPRAYING) without needing to know the "how" (the underlying Newton-Euler differential equations). Complex physics and communication protocols are simplified into easy-to-use method calls like step() and sendMessage().

Project Execution Flow:

1. System Initialization

- **Main Entry:** Main.java starts and calls the Simulator constructor.
- **Config Loading:** Simulator.java reads config.txt to set environment size, drone count, and physical properties (mass, drag).
- **Object Creation:** The simulator instantiates the CommunicationModule, FormationManager, CollisionAvoidance, and the individual Drone objects with their starting positions.
- **GUI Launch:** Main launches DroneGUI, which sets up the 3D JavaFX stage, camera, and lighting.

2. The Real-Time Simulation Loop

Once the GUI starts, an AnimationTimer triggers the simulation step approximately every 20ms. Each step follows this sequence:

1. **Sense (Communication):** Each drone uses the CommunicationModule to identify which other drones are within range and successfully transmitting data.
2. **Think (Swarm Logic):** * The FormationManager calculates forces to keep drones grouped.
 - CollisionAvoidance calculates repulsive forces if drones are too close.
 - Environment calculates forces to keep drones inside the mission boundaries.
3. **Act (Control):** The Controller compares the drone's current position to its mission target and calculates the required engine thrust and torque.
4. **Integrate (Physics):** Drone.java sums all forces (Thrust + Swarm + Gravity + Drag) and uses Newton-Euler integration to update the drone's 3D position and rotation.

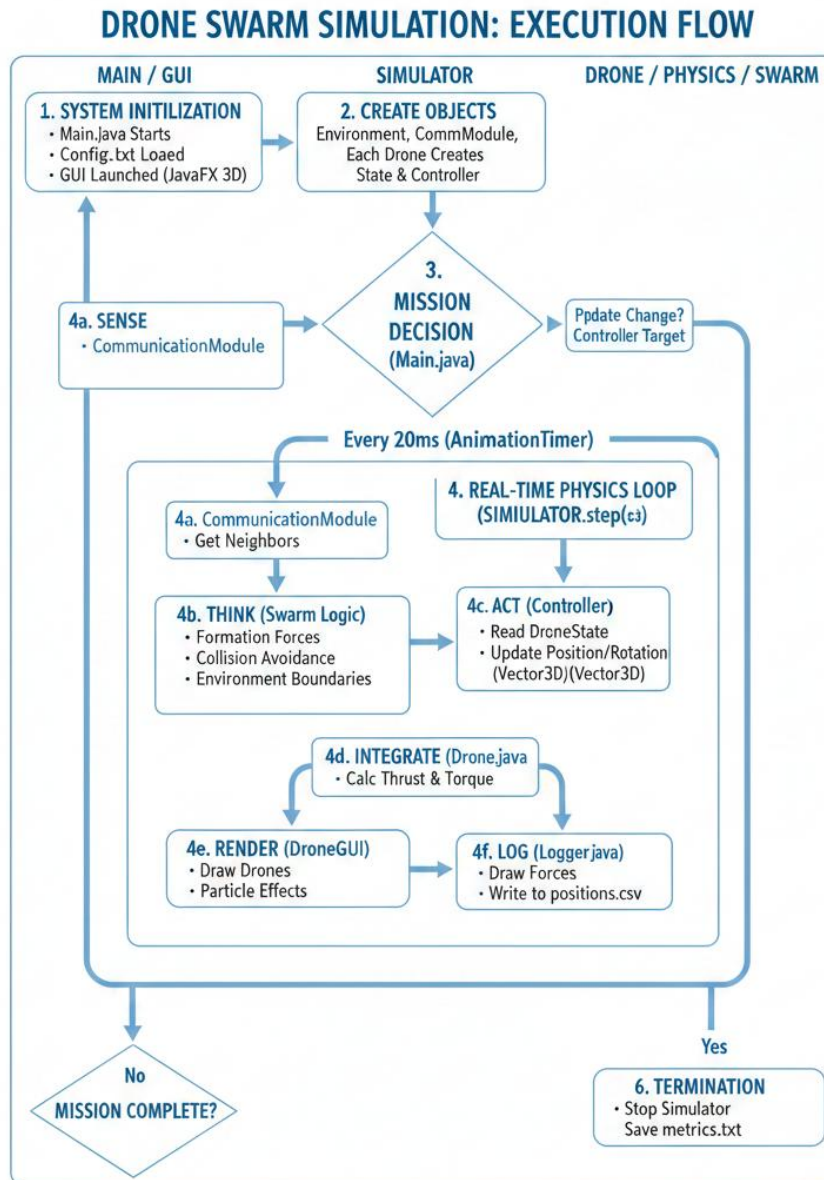
3. Mission Phase Progression

While the physics loop runs, Main.java monitors the progress of each drone using a state machine:

- **Phase 1: GO_TO_TARGET:** Drones steer toward the agricultural field coordinates.
- **Phase 2: SPRAYING:** Once within range of the target, drones hover and activate the spray particle system for 10 seconds.
- **Phase 3: RETURNING:** After spraying, the drones autonomously steer back toward their initial base coordinates.
- **Phase 4: DONE:** Drones reach the base and maintain a final stable position.

4. Data Logging and Termination

- **Real-time Recording:** Throughout the flight, Logger.java writes telemetry data (X, Y, Z coordinates and thrust) into positions.csv.
- **Finalization:** When the user closes the GUI window, the simulator stops, and the logger generates a metrics.txt file summarizing the total drones used and mission success.



Summary of Mathematical Formulas

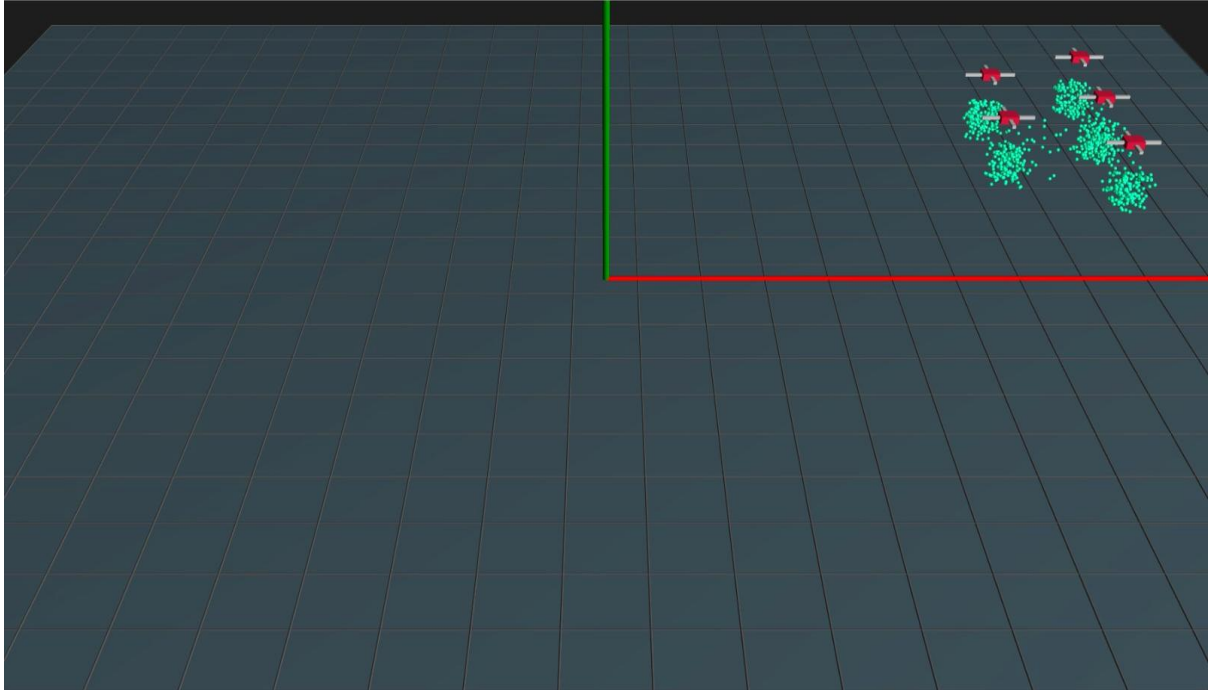
- **Translational Dynamics:** Models the net forces acting on the drone to determine linear acceleration based on Newton's Second Law (Force divided by Mass).
- **Rotational Dynamics:** Calculates angular acceleration by considering the applied torque and the drone's specific resistance to rotation (Inertia Tensor).
- **Position Integration:** Updates the drone's 3D position by adding the distance traveled (Velocity multiplied by the Time Step) to the previous location.
- **Velocity Integration:** Updates the current velocity by adding the change in speed (Acceleration multiplied by the Time Step).

- **Rotational Integration:** Updates the drone's orientation matrix to ensure the 3D model tilts and turns correctly in the visual environment.
- **Aerodynamic Drag:** Computes air resistance as a force that pushes directly against the drone's current direction of travel.
- **Control Acceleration:** Determines the necessary "push" to reach a target by measuring the error between the current position/velocity and the desired target.
- **Vertical Thrust:** Calculates the specific amount of upward force needed based on the drone's current tilt and total commanded power.
- **Torque Control:** Calculates the twisting force required to rotate the drone into a specific orientation to face its target.
- **Formation Force:** Synchronizes the swarm by calculating a combined force that pulls neighbors together while matching their flight speeds.
- **Collision Avoidance:** Generates a repulsive "push" that grows exponentially stronger as two drones get closer, ensuring they maintain a safe buffer.
- **Communication Success:** Uses a probability model to simulate real-world signal interference and determine if a message successfully reached a neighbor.
- **Communication Range:** Filters data exchange so that drones only interact with others located within a specific physical radius.
- **Average Spacing:** A metric used to track the health of the swarm by calculating the mean distance between all active agents.

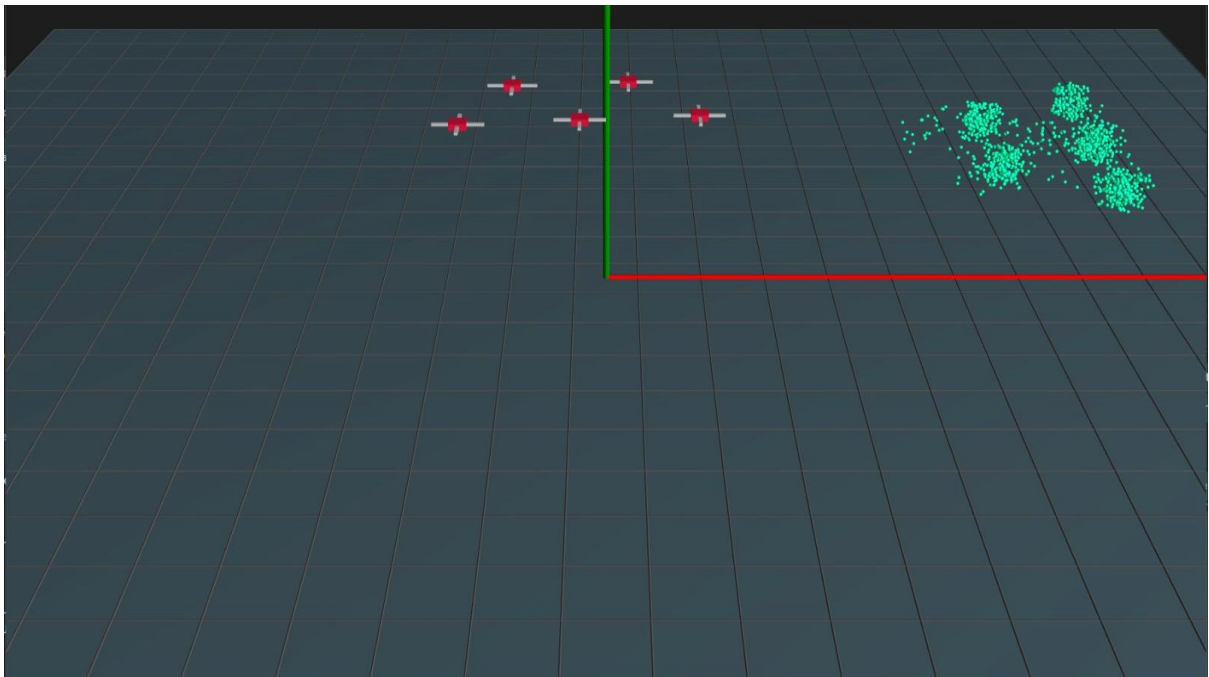
OUTPUT OF PROGRAM:

Spraying:

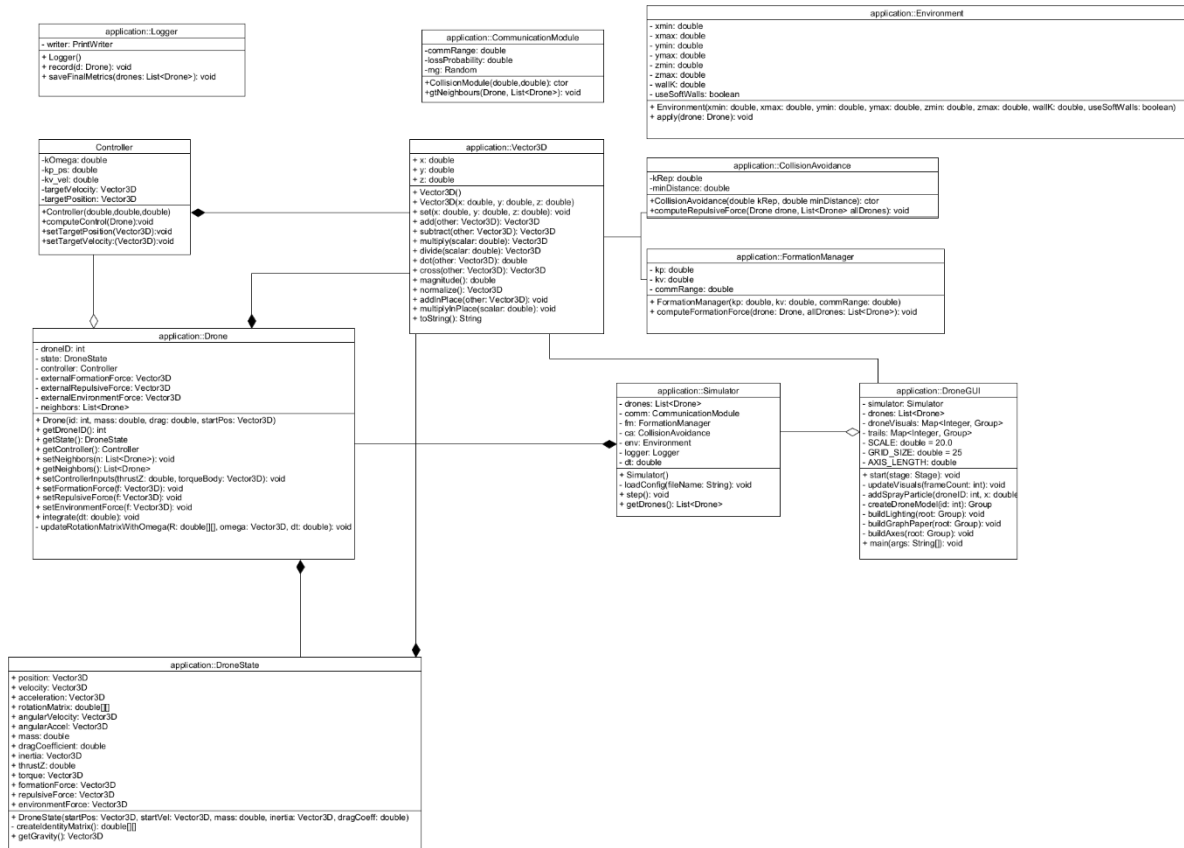
Below is the picture of simulation that shows the drones while spraying.



Beginning and Ending point:



UML Class Diagram:



Limitations:

1. Communication Realism

- **Binary Packet Loss:** The CommunicationModule uses a simple random probability to decide if a message is delivered or lost. In real-world scenarios, signal strength varies based on obstacles, multipath fading, and interference from other electronic devices, which is not modeled here.
- **Zero Latency:** The simulation assumes that if a message is successful, it arrives instantly. It does not account for the network delay (latency) that occurs in real-world swarm synchronization.

2. Physical and Environmental Constraints

- **Static Wind Conditions:** The physics engine in Drone.java accounts for gravity and drag, but it assumes a perfectly still atmosphere. There is no logic for wind gusts or turbulent air, which significantly impacts drone stability during outdoor spraying missions.

- **Simplified Obstacles:** The Environment class only manages world boundaries (xmin, xmax, etc.). It does not support internal 3D obstacles like trees, buildings, or power lines that the drones would need to navigate around.

3. Flight Dynamics & Hardware

- **Simplified PD Control:** The Controller class uses a translational PD loop and simple angular damping. It does not simulate the individual RPM of four separate motors, nor does it account for "Ground Effect" (the aerodynamic lift increase when flying very close to the ground).
- **Infinite Energy:** There is no battery or fuel consumption model. In a real mission, a drone's weight would decrease as it sprays pesticide, and it would eventually need to return to base due to low battery, regardless of the mission phase.
- **Ideal Sensors:** The drones have "perfect knowledge" of their own position and velocity from DroneState. Real drones rely on GPS and IMUs (Inertial Measurement Units), which have noise, drift, and periodic signal loss.

4. Swarm and Mission Logic

- **Hardcoded Targets:** The mission target (14, 0, 10) is hardcoded in Main.java. A more advanced system would allow for dynamic path planning to cover an entire field area (e.g., lawnmower patterns) rather than a single point.
- **Homogeneous Fleet:** Every drone is identical in mass, drag, and inertia. The code does not account for a "Heterogeneous Swarm" where different drones might carry different payloads or have different flight capabilities.

5. File I/O

- In our project program takes some input from file i.e. number of drones etc. but if we want to make it more efficient and add more functionality than we must add GUI in this case also to serve user reliability.

Conclusions:

This project successfully demonstrates the development of a physics-based autonomous drone fleet simulator designed for precision agriculture, proving that complex swarm intelligence can be effectively modeled through a modular Java architecture. By integrating **Newtonian physics** for realistic flight dynamics with a **state-machine mission logic**, the system successfully orchestrated a full lifecycle of takeoff, waypoint navigation, a 10-second spraying operation, and autonomous return-to-base. Key technical achievements include the implementation of a **PD control system** for stable flight, a **FormationManager** for group cohesion, and a **CollisionAvoidance** system that ensures safety through repulsive-force algorithms. While the current version relies on a static configuration-file input, the results

show that the combination of real-time 3D visualization and high-resolution data logging provides a powerful tool for analyzing multi-agent coordination. Ultimately, this simulation serves as a robust proof-of-concept for how autonomous swarms can enhance efficiency and safety in real-world agricultural applications.

References:

Video links to setup JavaFX library.

https://youtu.be/_7OM-cMYWbQ?si=i7qljKs6UARwgu4

<https://www.oracle.com/java/technologies/javase/javafx-overview.html>